Linux Challenge for DevOps Students

Objective:

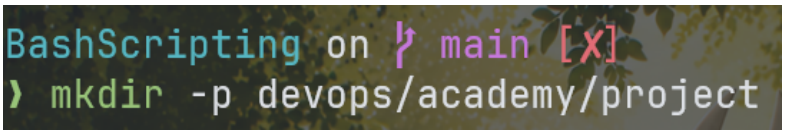
The objective of this challenge is to assess the students' ability to manage a Linux environment effectively, write shell scripts, and perform system administration tasks. They should be able to demonstrate their understanding of Linux file system hierarchy, user management, process management, networking, and automation.

Challenge Tasks:

Beginner Level

Task 1: Basic Linux Commands

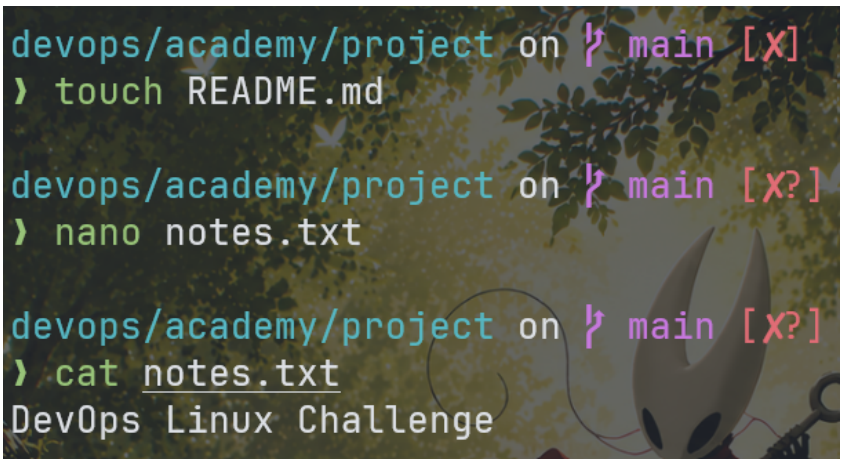
1. **Create and Navigate Directories:**
   - Create a directory structure as follows: `/home/devops/academy/project`.



   - Navigate into the `project` directory.



2. **File Management:**
   - Create an empty file named `README.md` inside the `project` directory.
   - Create another file named `notes.txt` and add the text "DevOps Linux Challenge" into it.
   - Copy `notes.txt` to `/home/devops/academy/backup/`.



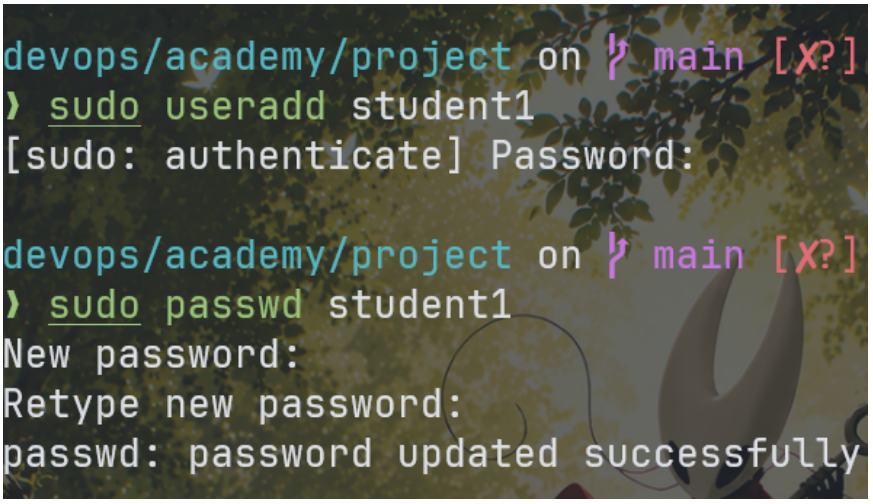


3. **Permissions:**
    - Change the permissions of `README.md` to be readable and writable only by the owner.
    - Make `notes.txt` readable by everyone, but writable only by the owner.



Task 2: User and Group Management

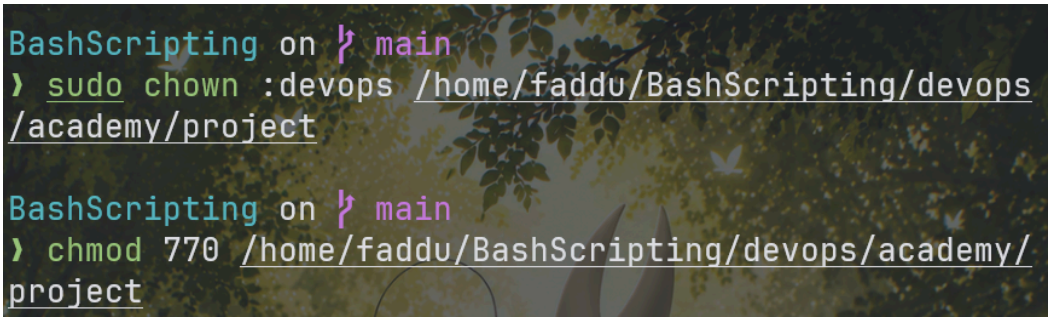
1. **Create Users:**
    - Create a new user named `student1`.
    - Set a password for `student1`.

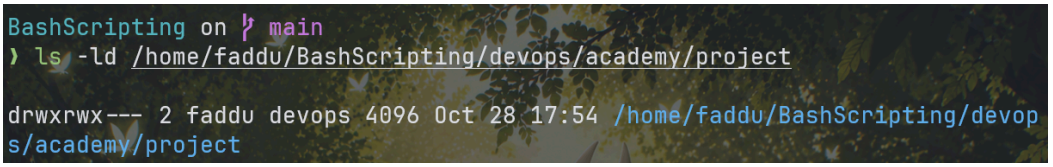


2. **Groups:**
    - Create a new group named `devops`.
    - Add `student1` to the `devops` group.

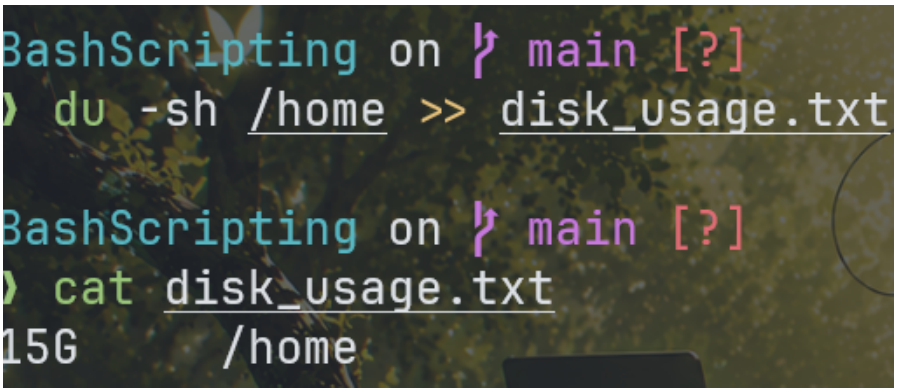


3. **User Permissions:**
    - Ensure that the `project` directory is accessible to members of the `devops` Group.





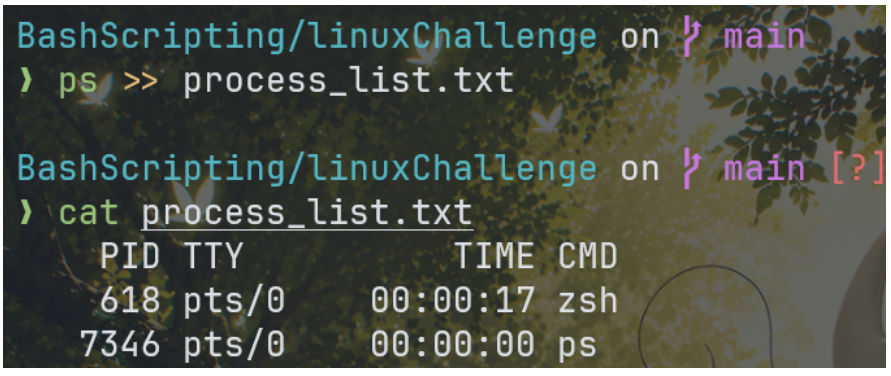
4. **Disk Usage:**
    - Display disk usage in human-readable format for `/home` directory and save the output to a file named `disk_usage.txt`.



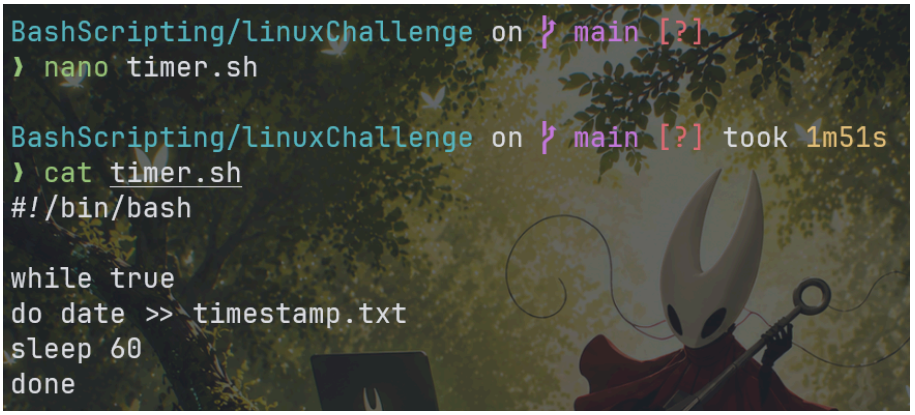
Intermediate Level

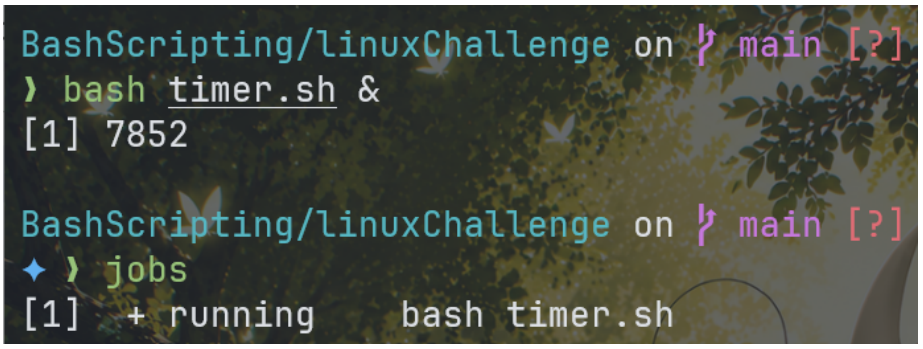
Task 3: Process Management

1. **List Processes:**
    - Display all running processes and redirect the output to a file named `Process_list.txt`.

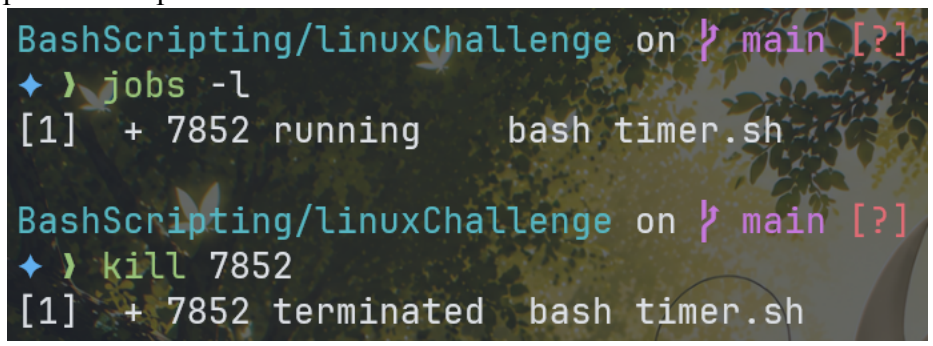


2. **Background Process:**
    - Start a simple background process that writes the current date and time to a file named `timestamp.txt` every minute using a `while` loop and `sleep`.

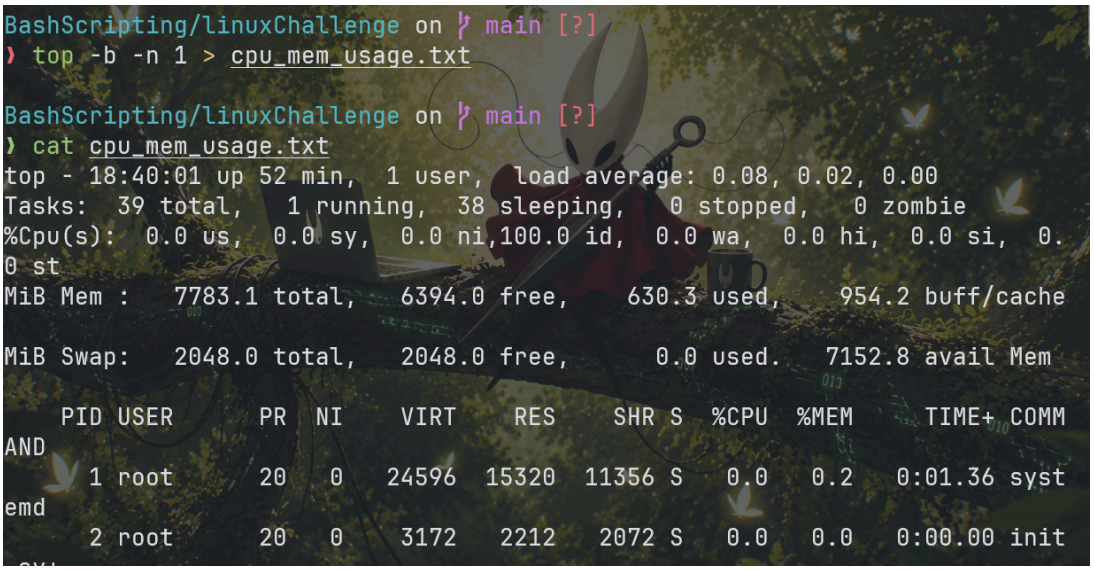




3. **Kill Process:**
    - Find the process ID (PID) of the background process started in the previous step and terminate it.

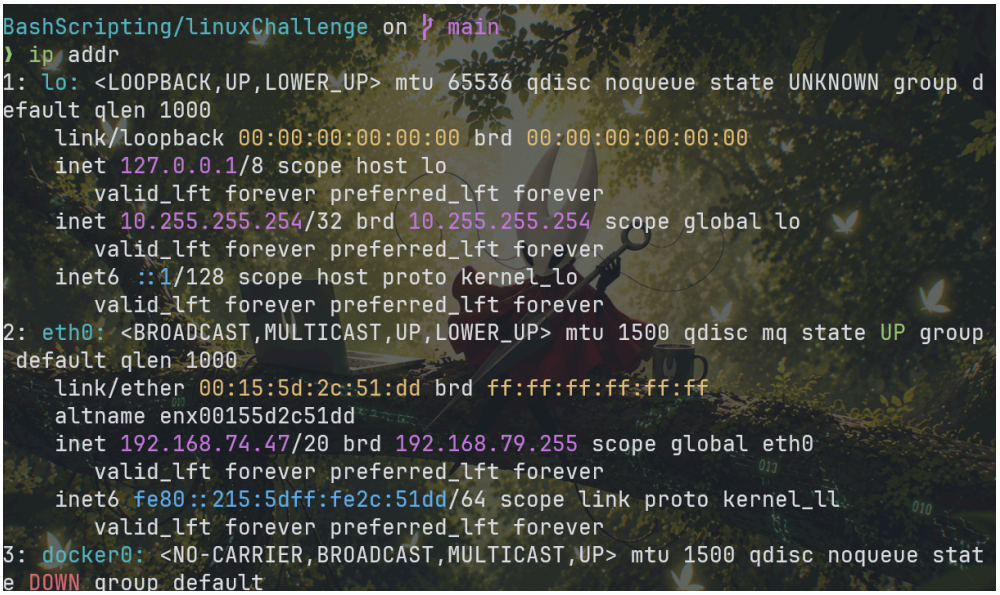


4. **CPU and Memory Usage:**
    - Display the current CPU and memory usage using `top` or `htop`, save a snapshot of this information to a file named `cpu_mem_usage.txt`.

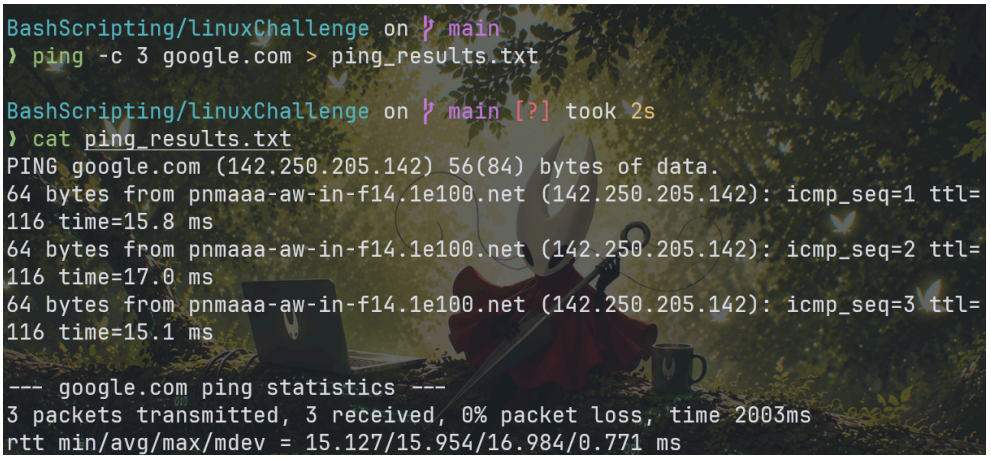


Task 4: Networking

1. **Network Configuration:**
   - Display the current network configuration using `ifconfig` or `ip addr`.



2. **Ping Test:**
   - Ping `google.com` and save the output to a file named `ping_results.txt`.

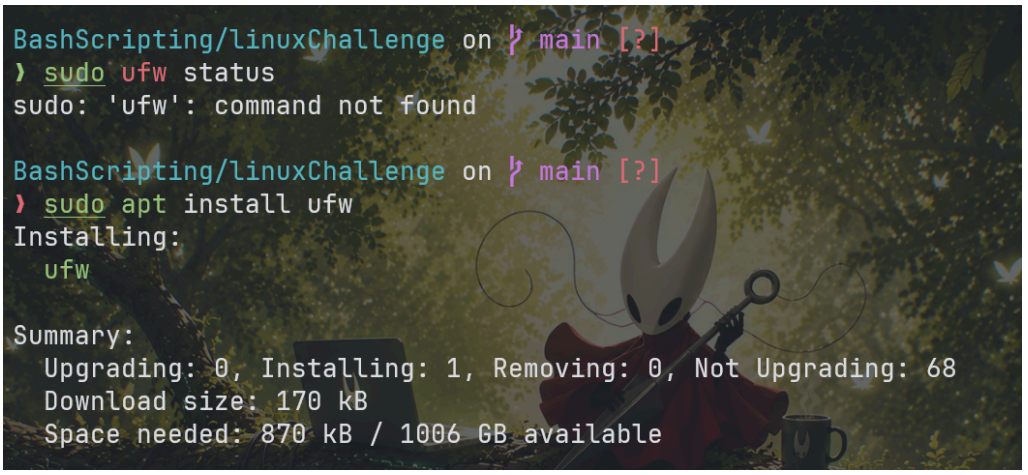


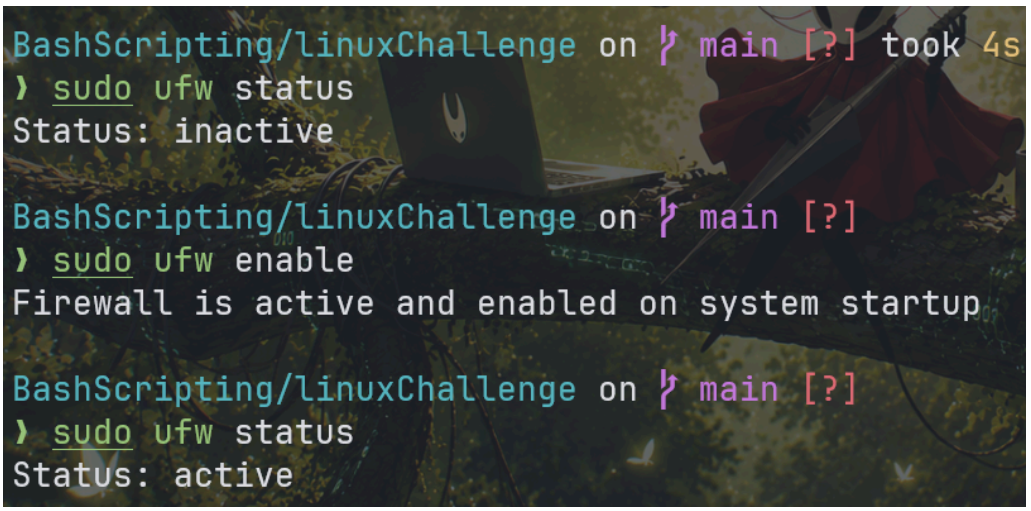
3. **Open Ports:**
   - List all open ports on the system using `netstat` or `ss` command.



4. **Firewall Configuration:**

- Check if the `ufw` (Uncomplicated Firewall) is installed and running. If not, install and enable it.





- Allow incoming connections on port 80 (HTTP) and 443 (HTTPS).



Task 5: Shell Scripting

1. **Backup Script:**
   - Write a shell script named `backup.sh` that compresses the `project` directory into a tar.gz file and saves it in the `/home/devops/academy/backup` directory. The script should include error handling to ensure the backup only proceeds if the `project` directory exists.

```
BashScripting/linuxChallenge on  main [?]
❯ nano backup.sh

BashScripting/linuxChallenge on  main [?] took 8m20s
❯ cat backup.sh
#!/bin/bash

PROJECT_DIR="/home/faddu/BashScripting/linuxChallenge/devops/academy/project"
BACKUP_DIR="/home/faddu/BashScripting/linuxChallenge/devops/academy/backup"

if [ ! -d "$PROJECT_DIR" ]; then
    echo "Error: Project directory does not exist: $PROJECT_DIR"
    exit 1
fi

if [ ! -d "$BACKUP_DIR" ]; then
    mkdir -p "$BACKUP_DIR" || {
        echo "Error: Could not create backup directory."
        exit 1
    }
fi

if tar -czvf "$BACKUP_DIR/project_backup.tar.gz" "$PROJECT_DIR"; then
    echo "Backup successful"
else
    echo "Error: Backup failed."
    exit 1
fi
```

```
BashScripting/linuxChallenge on  main [?]
❯ bash backup.sh
tar: Removing leading `/' from member names
/home/faddu/BashScripting/linuxChallenge/devops/academy/project/
/home/faddu/BashScripting/linuxChallenge/devops/academy/project/README.md
/home/faddu/BashScripting/linuxChallenge/devops/academy/project/notes.txt
Backup successful

BashScripting/linuxChallenge on  main [?]
❯ ls devops/academy/backup
notes.txt  project_backup.tar.gz
```

2. **Automation Script:**
   - Create a script named `cleanup.sh` that deletes all files in the `/home/devops/academy/backup` directory that are older than 7 days. Schedule this script to run daily using `cron`.

```
BashScripting/linuxChallenge on  main [?]
❯ nano cleanup.sh

BashScripting/linuxChallenge on  main [?] took 20s
❯ cat cleanup.sh
#!/bin/bash

BACKUP_DIR="/home/faddu/BashScripting/linuxChallenge/devops/academy/backup"

if [ ! -d "$BACKUP_DIR" ]; then
    echo "Error: Backup directory does not exist"
    exit 1
fi

find "$BACKUP_DIR" -type f -mtime +7 -delete

echo "Cleanup completed"
```

```
For example, you can run a backup of all your user accounts
at 5 a.m every week with:
0 0 * * * /bin/bash /home/faddu/BashScripting/linuxChallenge/devops/academy/cleanup.sh
```

3. **Log Rotation:**
    - Write a script named `rotate_logs.sh` to rotate the `timestamp.txt` file if it exceeds 1MB in size and save old logs with a timestamp in the filename.

```
BashScripting/linuxChallenge on  main [?]
❯ nano rotate_logs.sh

BashScripting/linuxChallenge on  main [?] took 3m46s
❯ cat rotate_logs.sh
#!/bin/bash

FILE="/home/faddu/BashScripting/linuxChallenge/timestamp.txt"

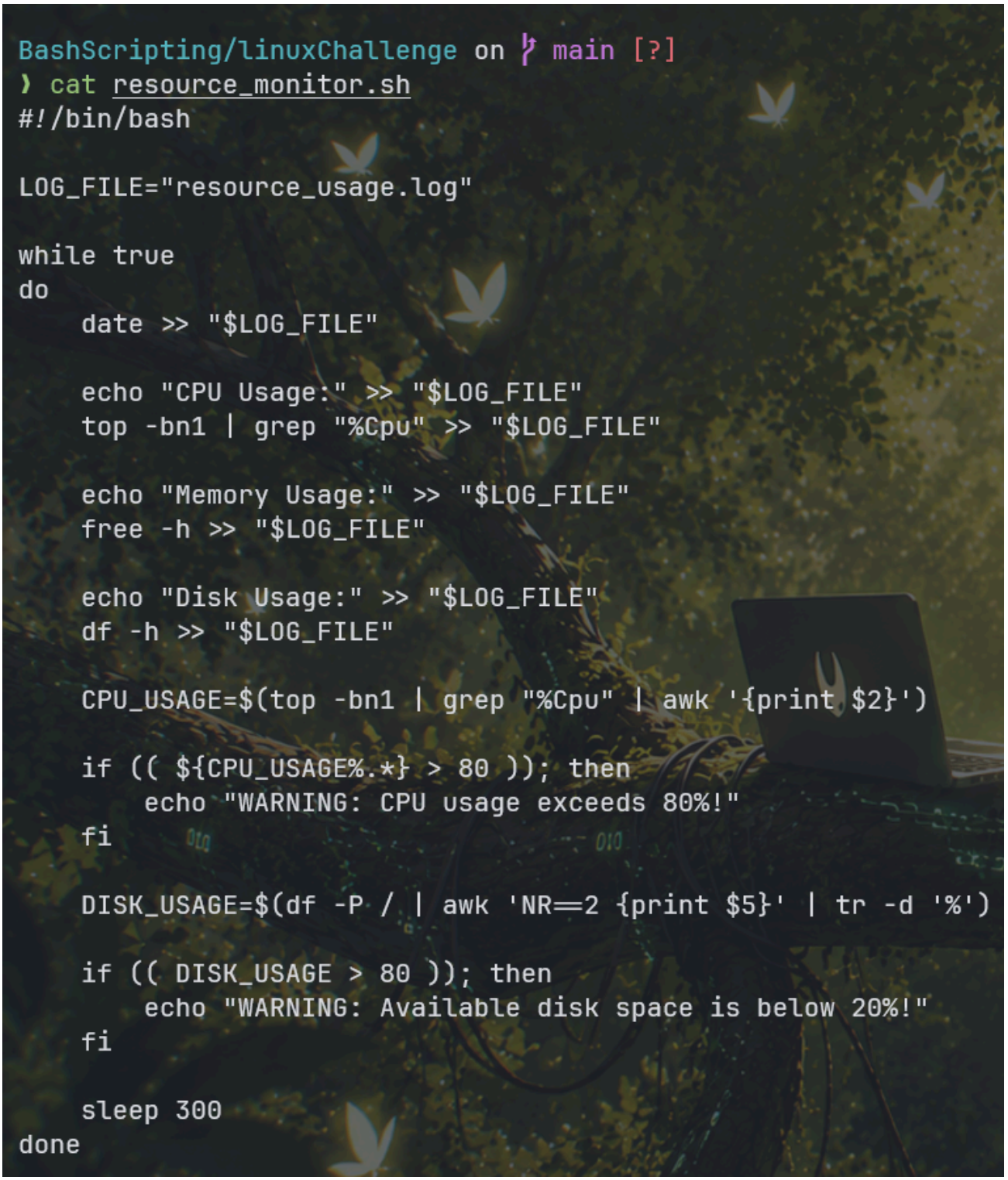
if [ -f "$FILE" ] && [ "$(wc -c < "$FILE")" -gt 1048576 ]; then
    mv "$FILE" "${FILE}_$(date +%Y%m%d_%H%M%S)"
    touch "$FILE"
    echo "Log rotated successfully"
else
    echo "File does not exist or is not larger than 1MB"
fi
```

```
BashScripting/linuxChallenge on  main [?]
❯ bash rotate_logs.sh
File does not exist or is not larger than 1MB
```

Advanced Level:

Advanced Shell Scripting:

- Write a script named `resource_monitor.sh` that:
  - Monitors CPU, memory, and disk usage.
  - Logs usage data every 5 minutes to a file named `resource_usage.log`.
  - Sends an alert email if CPU usage exceeds 80% or available disk space falls below 20%.



```
BashScripting/linuxChallenge on  main [?]
❯ cat resource_monitor.sh
#!/bin/bash

LOG_FILE="resource_usage.log"

while true
do
    date >> "$LOG_FILE"

    echo "CPU Usage:" >> "$LOG_FILE"
    top -bn1 | grep "%Cpu" >> "$LOG_FILE"

    echo "Memory Usage:" >> "$LOG_FILE"
    free -h >> "$LOG_FILE"

    echo "Disk Usage:" >> "$LOG_FILE"
    df -h >> "$LOG_FILE"

    CPU_USAGE=$(top -bn1 | grep "%Cpu" | awk '{print $2}')

    if (( ${CPU_USAGE%.*} > 80 )); then
        echo "WARNING: CPU usage exceeds 80%!"
    fi

    DISK_USAGE=$(df -P / | awk 'NR==2 {print $5}' | tr -d '%')

    if (( DISK_USAGE > 80 )); then
        echo "WARNING: Available disk space is below 20%!"
    fi

    sleep 300
done
```

User Management Automation:

- Write a script named `bulk_user_add.sh` that:
  - Reads a list of usernames from a file named `user_list.txt`.
  - Creates each user and sets a default password.
  - Adds each user to a specific group (e.g., `students`).
  - Ensures each user has a home directory created.

```
BashScripting/linuxChallenge on  main [?]
❯ cat bulk_user_add.sh
#!/bin/bash

USER_FILE="user_list.txt"
GROUP="students"
DEFAULT_PASSWORD="Student@123"

if [ ! -f "$USER_FILE" ]; then
    echo "Error: $USER_FILE not found."
    exit 1
fi

if ! getent group "$GROUP" > /dev/null; then
    sudo groupadd "$GROUP"

    if [ $? -ne 0 ]; then
        echo "Error: Failed to create group $GROUP."
        exit 1
    fi
fi

while read username
do
    if [ -z "$username" ]; then
        continue
    fi

    if id "$username" &>/dev/null; then
        echo "User $username already exists. Skipping."
        continue
    fi

    sudo useradd -m -g "$GROUP" "$username"

    if [ $? -ne 0 ]; then
        echo "Error: Failed to create user $username."
        continue
    fi
```

```
    echo "$username:$DEFAULT_PASSWORD" | sudo chpasswd

    if [ $? -ne 0 ]; then
        echo "Error: Failed to set password for $username."
        continue
    fi

    sudo passwd -e "$username"

    echo "User $username created successfully."

done < "$USER_FILE"

echo "Bulk user creation completed."
```